SECURESCAN

SÉCURISEZ VOTRE CODE EN UN CLIC

AZAG Dillon
MARGARIAN Diana
NGUYEN Thi Van Anh

PLAN

Contexte & Problématique

Solution proposée – SecureScan

Architecture & Choix techniques

Fonctionnalités clés

Démonstration

Conclusion & Perspectives

LE CONTEXTE & LE PROBLÈME

CyberSafe Solutions souhaite détecter automatiquement les vulnérabilités du code source.

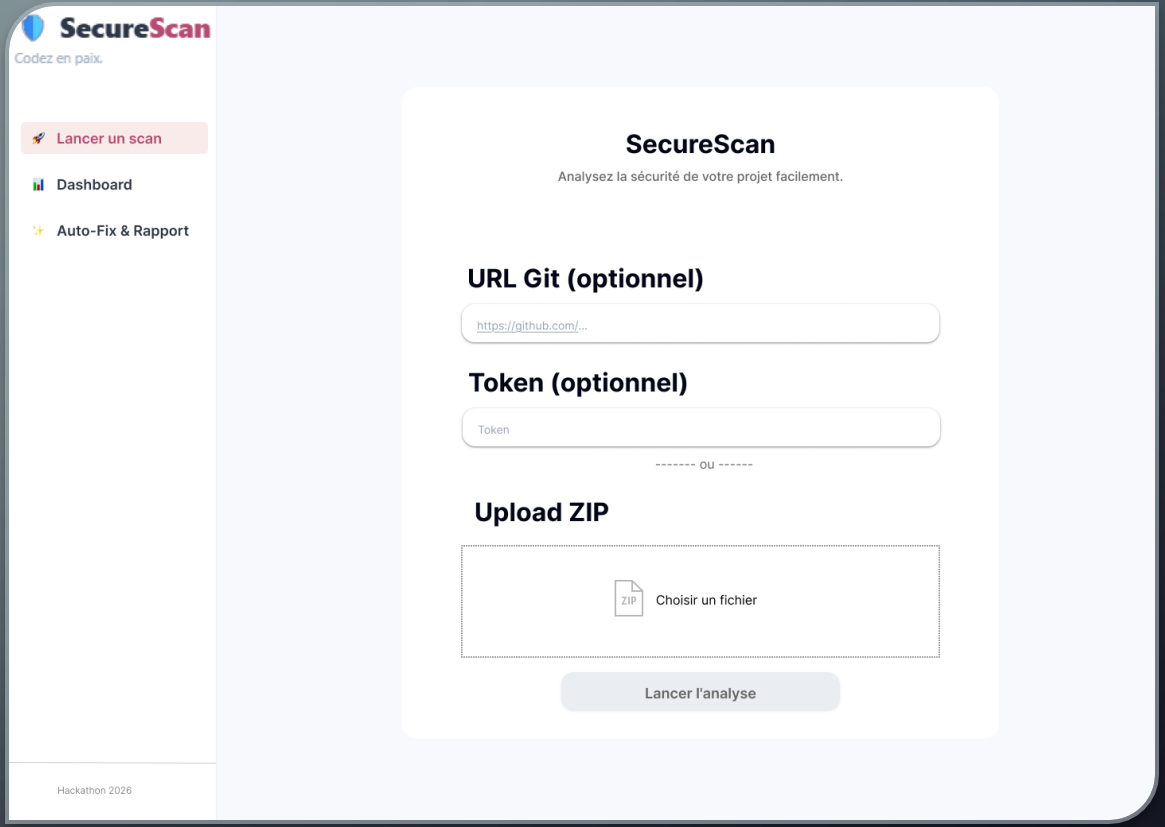
Problèmes actuels :

- Aucun scan sécurité centralisé
- Failles injection non détectées avant production
- Dépendances vulnérables
- Corrections manuelles lentes

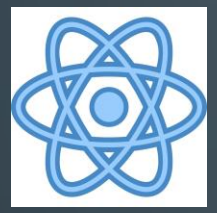
OBJECTIF DU PROJET

Développer une plateforme qui permet :

- Analyse sécurité automatisée
- Mapping vers OWASP Top 10
- Système de correction assistée
- Génération de rapport



CHOIX TECHNOLOGIQUES



Expérience utilisateur fluide

- Performance élevée
- Support natif de l'asynchrone

- Déploiement rapide
- ORM sécurisé
- Fiabilité

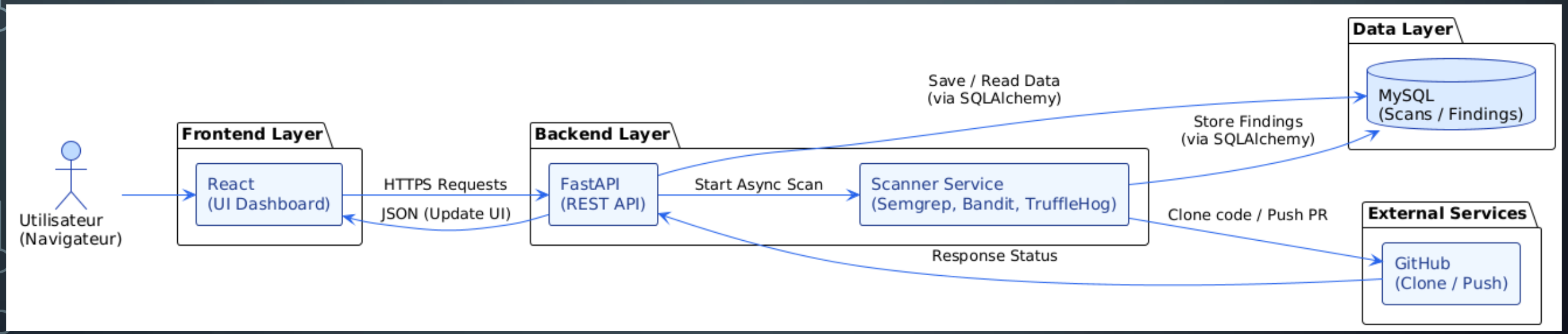
LES MOTEURS D'ANALYSE

SAST

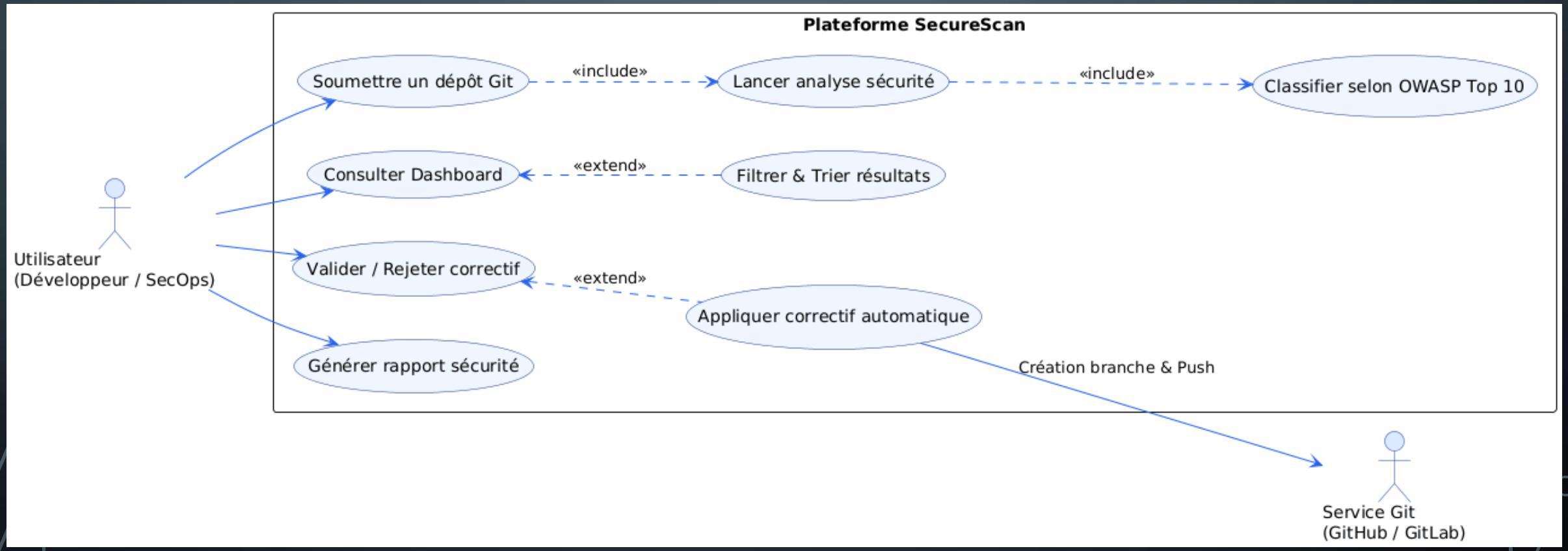
Qualité - Python

TruffleHog - Secrets

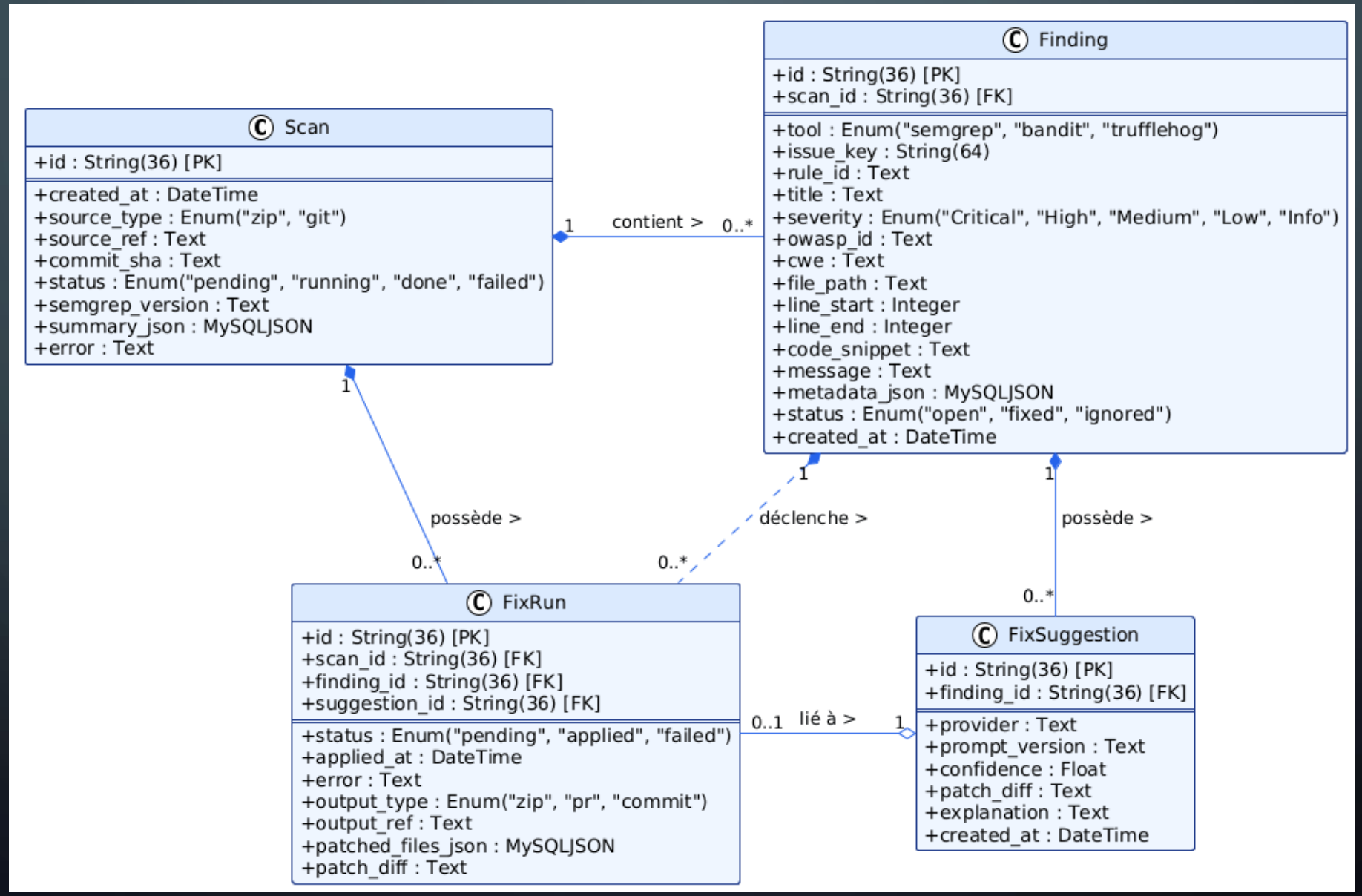
ARCHITECTURE GLOBALE



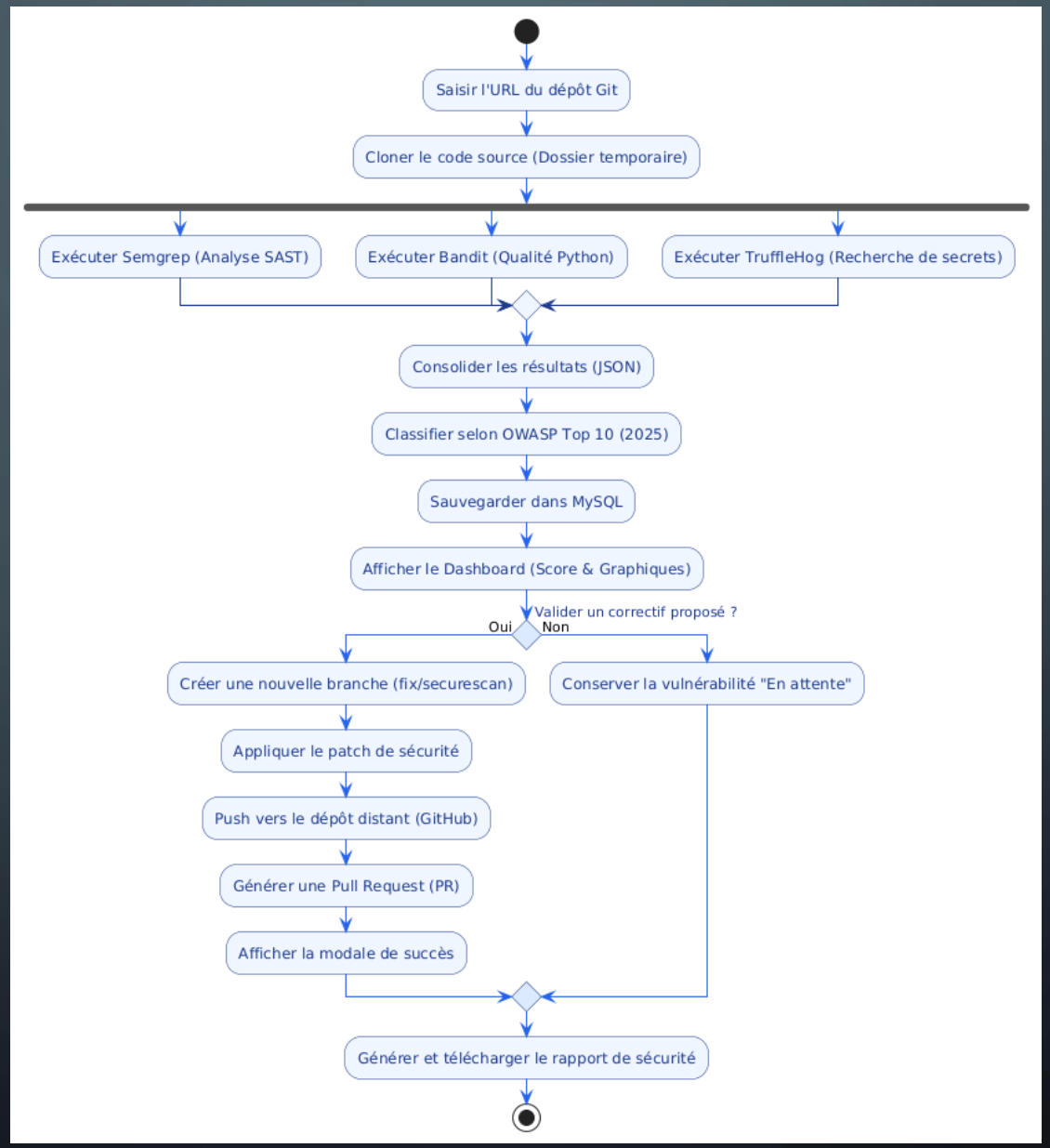
USE CASE


Plateforme SecureScan
Soumettre un dépôt Git
«include»
Lancer analyse sécurité
«include»
Classifier selon OWASP Top 10
Consulter Dashboard
«extend»
Filtrer & Trier résultats
Utilisateur
(Développeur / SecOps)
Valider / Rejeter correctif
«extend»
Appliquer correctif automatique
Générer rapport sécurité
Création branche & Push
Service Git
(GitHub / GitLab)

MODÈLE DE DONNÉES



WORKFLOW D'EXÉCUTION

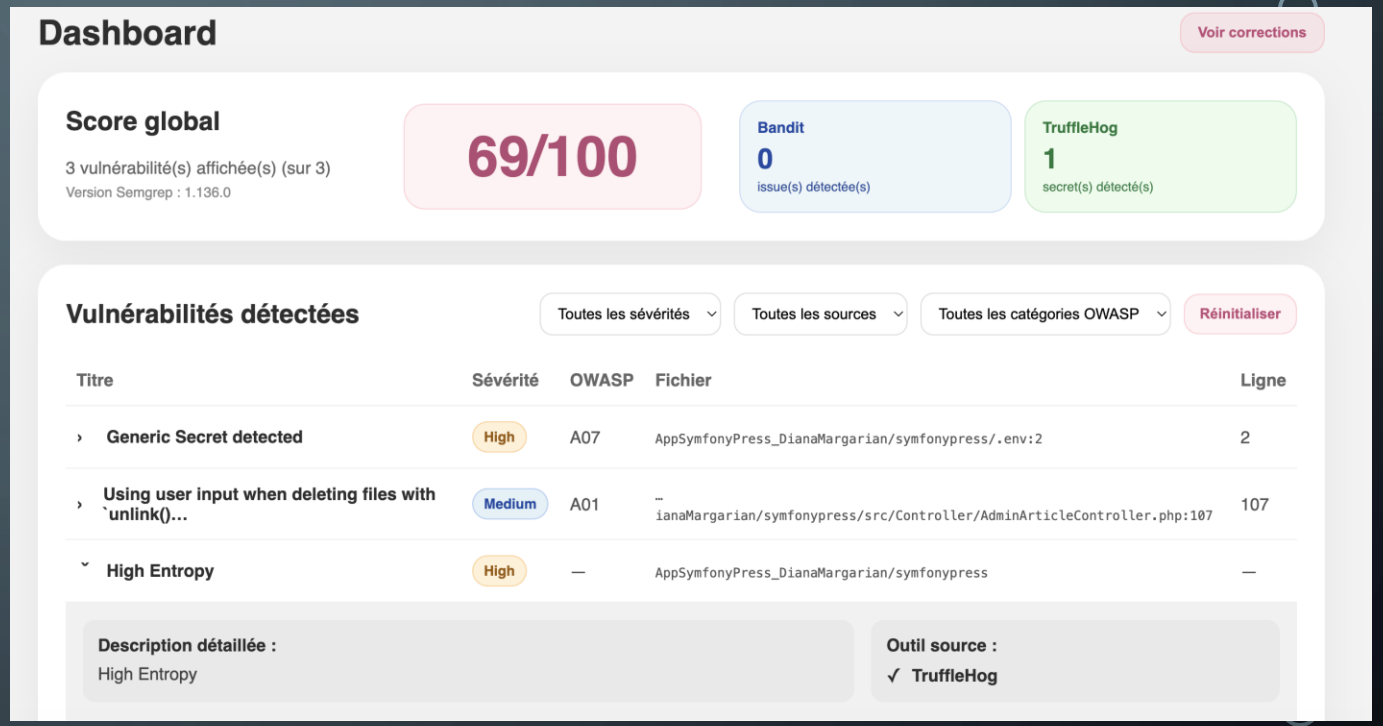


CLASSIFICATION OWASP TOP 10

Outil	Détection brute	Catégorie OWASP 2025
Semgrep	Faille SQL (CWE-89) / XSS	A05:2025 - Injection
Semgrep	Faille SSRF / Contrôle d'accès	A01:2025 - Broken Access Control
Bandit	Mots de passe codés en dur	A07:2025 - Authentication Failures
Bandit	Blocs try/except vides (Fail-open)	A10:2025 - Mishandling of Exceptional Conditions
TruffleHog	API Keys / Secrets exposés dans Git	A02:2025 - Security Misconfiguration

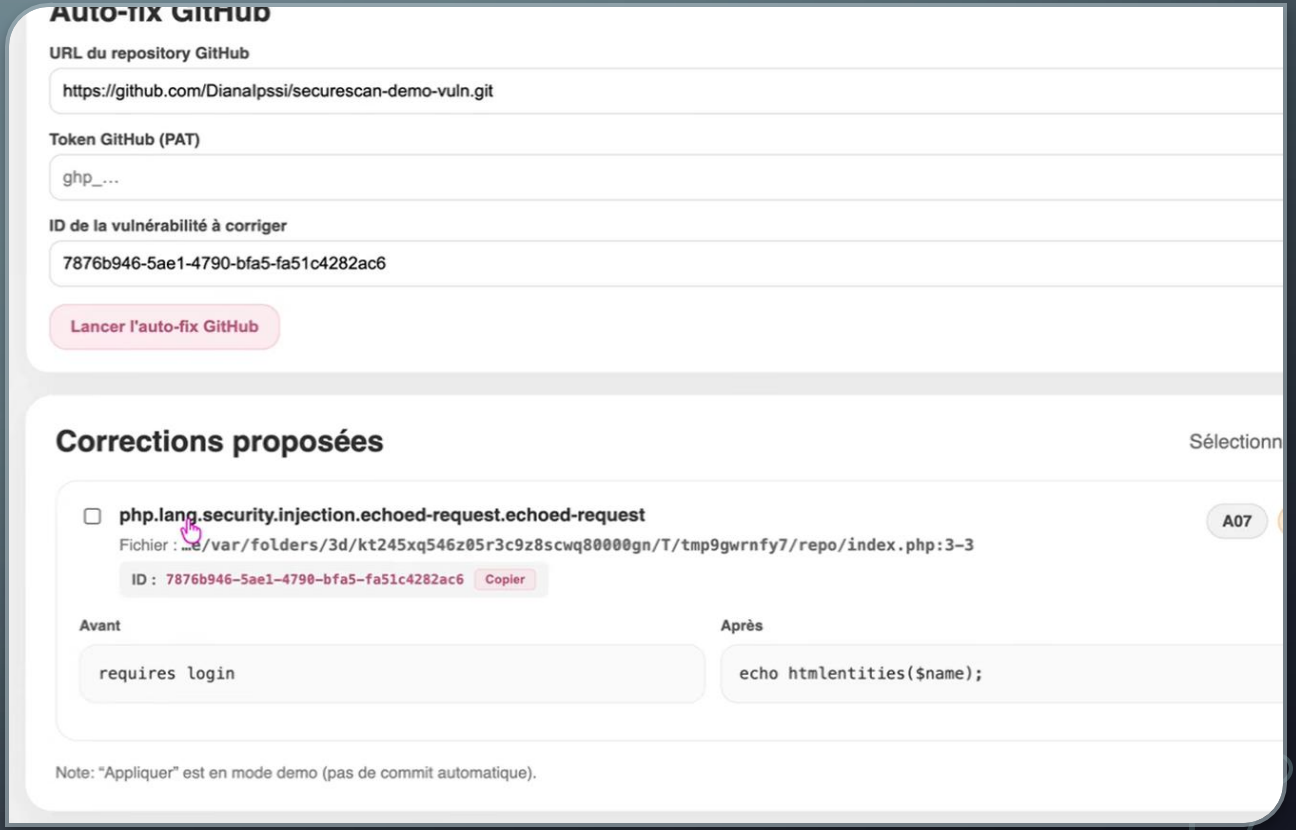
LE DASHBOARD

- 🛡️ **Score** de sécurité global
- ⚠️ Vulnérabilités **classées** par **gravité**
- 📊 Distribution selon OWASP Top 10
- 🔍 Consultation **détaillée** avec **filtres**

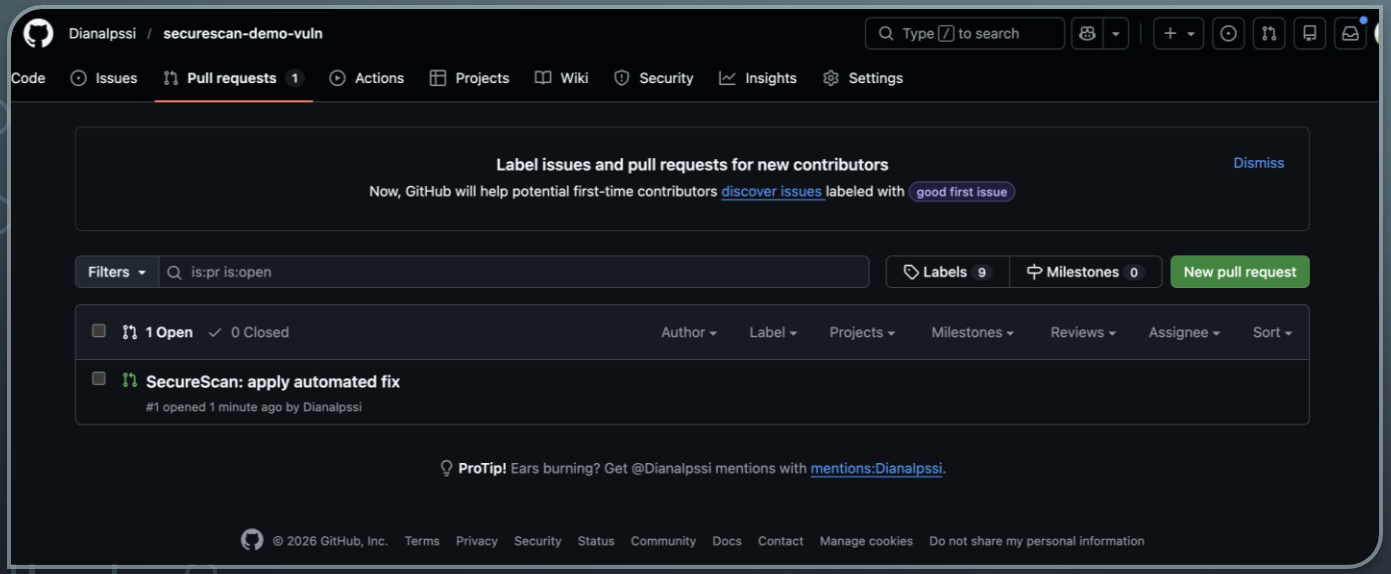


LE CŒUR DU PROJET : L'AUTO-FIX

- 🔍 Détection des vulnérabilités
- 🛡️ **Correctif automatique** sécurisé
- ⚖️ Comparaison **avant / après**
- ✅ **Validation utilisateur**



INTÉGRATION GIT AUTOMATISÉE



Clic "Auto-Fix"

🛠 Application des correctifs

🌿 **Création Branche Git**

🚀 **Génération Pull Request**

GÉNÉRATION DU RAPPORT

- 📄 Génération **automatique** d'un rapport de sécurité
- 📊 **Résumé** de l'analyse
- ⬇ Téléchargement **PDF/HTML**

SecureScan Report

Scan ID: c70d616f-4b23-[redacted]
Date: 2026-03-[redacted]
Source: .git — https://github.com/thivananhnguyen/my_blog.git
Status: done

Summary

Severity	Count
Critical	0
High	1
Medium	1
Low	0
Info	0

Total findings: 2

OWASP

OWASP ID	Count
A01:2025	1
A07:2025	1

Findings

Tool	Severity	Title	File	Line	Rule	OWASP	St
semgrep	Medium	The `redirect()` method does not check its destination in any way. If you redirect to a URL provided by end-users, your application may be open to the unvalidated redirects security vulnerability. Consider using literal values or an allowlist to validate URLs.	src/Controller/Admin/DashboardController.php	26	php.symfony.security.audit.symfony-non-literal-redirect.symfony-non-literal-redirect	A01:2025	op
semgrep	High	Generic Secret detected	.env.dev	3	generic.secrets.security.detected-generic-secret.detected-generic-secret	A07:2025	op

SÉCURITÉ "BY DESIGN"

- 🔗 **Input Validation** : Nettoyage des URLs (strip/lower) et blocage des dossiers sensibles.
- 📄 **CLI Secure** : Appels systèmes directs sans shell=True (Anti-Injection).
- 📂 **Temp Folder (Isolation)** : Auto-destruction du code source cloné après chaque scan.
- ⚡ **Anti-Surcharge** : Clonage Git superficiel (--depth 1).
- 🔑 **Secret Protection** : Variables .env cachées et utilisation d'un ORM

IMPACT BUSINESS

- ⏱️ Gain de **temps** (analyse automatique)
- 📉 **Réduction** des risques
- ✅ Amélioration conformité **sécurité**

DÉMO LIVE

FONCTIONNALITÉ FUTUR

Mistral AI

MERCI POUR VOTRE ATTENTION !

SECURESCAN : CODEZ EN PAIX